DEXARENA: A UNIFIED BENCHMARK FOR LONG-HORIZON DEXTEROUS MANIPULATION.

Anonymous authors
Paper under double-blind review

ABSTRACT

Long-horizon dexterous manipulation remains a fundamental challenge for robot learning, requiring the composition of multiple skills, precise bimanual coordination, generalizable visual understanding, and robust skill transitions. Yet existing manipulation benchmarks largely emphasize isolated skills or support only a limited range of end-effectors, making these capabilities difficult to evaluate systematically. We introduce **DexArena**, a benchmark designed to evaluate learning and generalization in long-horizon bimanual dexterous manipulation. DexArena features **10 long-horizon tasks and 30 stage-level tasks** spanning diverse household and industrial scenarios. Each long-horizon task is composed of explicitly defined stages with physically grounded success conditions, enabling reliable evaluation of both intermediate progress and end-to-end task completion. A unified benchmark interface and verified human-teleoperated demonstrations enable testing different policy learning algorithms, with standardized simulation configurations, scene randomization, trajectory replay, and task-level evaluation. Through extensive baseline experiments, we identify substantial challenges in precise manipulation, generalization to scene variation, and skill composition. DexArena establishes a challenging and reproducible benchmark for developing robot learning methods that move toward generalizable long-horizon dexterous behavior.

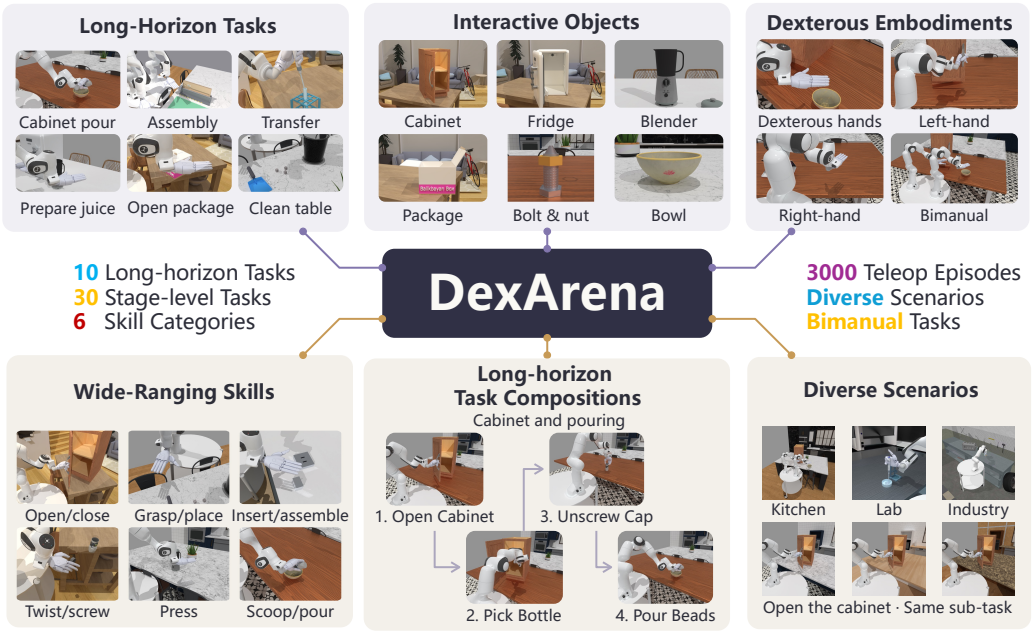


Figure 1: **DexArena overview.** A benchmark for long-horizon bimanual dexterous manipulation with multi-stage tasks, diverse scenarios, teleoperated demonstrations, and multiple embodiments.

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

1 INTRODUCTION

Data-driven robot learning has made gripper-based manipulation increasingly capable. By learning from expert demonstrations, imitation learning methods (Zhao et al., 2023; Chi et al., 2023) can acquire a wide range of manipulation skills, such as grasping, placing, and plugging. However, much of this progress has focused on parallel-jaw end-effectors and short, isolated tasks. Dexterous hands substantially expand the space of behaviors that robots can perform: multiple fingers can regrasp, twist objects, and manipulate tools. These contact-rich, in-hand interactions are often beyond the capabilities of a parallel-jaw gripper. At the same time, dexterous manipulation introduces a high-dimensional action space and complex contact dynamics, making learning robust policies for long-horizon tasks particularly challenging.

Recent progress in this area also requires an efficient and reproducible evaluation infrastructure. Real-robot experiments are costly, time-consuming to reset, and difficult to reproduce across hardware platforms. These challenges are especially pronounced for dexterous hands, whose complex mechanisms and precise actuation can limit the duration and scale of repeated experiments. Physics-based simulators provide an attractive alternative by enabling controlled randomization, scalable interaction, and exact state reset. However, beyond individual simulated tasks, there is a need for a benchmark that systematically evaluates whether a learned policy can execute dexterous skills, compose them across multiple dependent stages, and generalize across different scene configurations.

The existing benchmark landscape leaves this problem only partially addressed. RLBench (James et al., 2020), Meta-World (Yu et al., 2020), ManiSkill2 (Gu et al., 2023), RoboCasa (Nasiriany et al., 2024), and RoboTwin (Mu et al., 2025) provide broad coverage of manipulation skills, household scenarios, or dual-arm settings, but primarily emphasize gripper-based manipulation. Dexterous benchmarks, including Adroit (Rajeswaran et al., 2018), DexArt (Bao et al., 2023), DexMimicGen (Jiang et al., 2025), and DexJoCo (Wang et al., 2026), have begun to address dexterous hand manipulation, even for bimanual tasks. However, long-horizon composition across dependent stages is rarely evaluated under a shared task definition. Consequently, it remains difficult to disentangle whether a method has learned a dexterous skill itself, can reliably compose skills over long horizons, or is specialized to a particular visual configuration.

We introduce DexArena, a unified benchmark designed to make these capabilities measurable within a common physical framework. DexArena contains 10 multi-stage tasks and 30 stage-level tasks spanning household and industrial-style scenarios. Each multi-stage task consists of two to five stages, while each stage corresponds to an exact slice of its parent sequence: the multi-stage task and its stages share the same workstation, dynamics, and randomization ranges, with identical success conditions for each corresponding stage. This design enables controlled evaluation of isolated skill execution and skill composition without changing the underlying physical setting. Task completion is determined by explicit criteria, including articulated-object angles, relative poses, and insertion depths. Success must persist for a specified duration, preventing transient contacts or unstable configurations from being counted as successful completion.

DexArena further separates skill learning from embodiment and rendering choices. The benchmark provides a common interface across a parallel-jaw gripper and three dexterous hands (Sharpa Hand, Wuji Hand, and XHand), and supports both single-arm and bimanual manipulation. All environments use standardized physical parameterizations and workstation configurations. Every recorded frame stores the scene state, so an episode can be restored frame by frame for re-rendering, and its recorded actions can be replayed from the recorded initial state. Importantly, recorded trajectories can also be rendered under different camera configurations and background scenes after collection, enabling controlled evaluation of visual generalization without changing the underlying physical interaction. To support data-driven learning, we collect human-teleoperated demonstrations for every stage-level task, providing physically valid trajectories for later evaluations.

Using this infrastructure, we evaluate three representative policy families: demonstration-guided reinforcement learning, single-task imitation learning, and Vision-Language-Action (VLA) models. We measure stage-level task success, generalization to scene variations, and end-to-end completion under sequential stage execution. These evaluations expose different failure modes, distinguishing failures in contact-rich skill execution, visual generalization, and long-horizon skill composition. Together, DexArena provides a common empirical framework for studying the transition from isolated manipulation skills to robust, compositional, and long-horizon dexterous behavior.

We summarize our contributions as follows:

1. We introduce DexArena, a unified benchmark for long-horizon, bimanual dexterous manipulation, with 10 multi-stage tasks and 30 physically grounded stage-level tasks that support evaluation of skill execution, generalization and composition.
2. We provide a reproducible benchmark infrastructure with four heterogeneous end-effectors, standardized simulation and randomization configurations, trajectory replay, and stage-level/task-level success criteria. In particular, we invest substantial effort in collecting and validating physically valid, replayable demonstrations, enabling consistent reproduction of the demonstrated interactions and providing high-quality data for policy learning.
3. We establish representative baselines spanning imitation learning, demonstration-guided reinforcement learning, and vision-language-action models, and evaluate them on stage-level task performance, scenario generalization, and long-horizon composition to provide a common reference point for future methods.

2 Related Work

Most existing manipulation benchmarks focus on gripper-based manipulation and have established the foundations for systematic evaluation of robot manipulation. RLBench (James et al., 2020) provides procedurally generated vision-based tasks and demonstrations. Meta-World (Yu et al., 2020) evaluates multi-task and meta-reinforcement learning across a family of tabletop skills. ManiSkill2 (Gu et al., 2023) provides GPU-parallel simulation, diverse assets, and standardized interfaces for large-scale manipulation experiments. RoboCasa (Nasiriany et al., 2024) extends evaluation to household manipulation through procedurally generated scenes and language-conditioned tasks. More recent benchmarks, including RoboTwin (Mu et al., 2025) and RoboDojo (Chen et al., 2026), further explore scalable data generation and evaluation for bimanual and gripper-based manipulation. Together, these benchmarks demonstrate the value of standardized simulation for studying visual generalization, multi-task learning, and manipulation policy performance.

Dexterous hands introduce additional challenges including underactuated or high-dimensional control, contact-rich interactions, and coordination across multiple fingers. Adroit (Rajeswaran et al., 2018) studies object relocation, in-hand manipulation, tool use, and door opening with a dexterous hand, using human demonstrations to support policy learning. DexArt (Bao et al., 2023) focuses on articulated-object manipulation with dexterous hands. DexMimicGen (Jiang et al., 2025) presents nine bimanual dexterous-manipulation environments across three embodiments and synthesizes demonstrations from a small set of human teleoperation demonstrations. DexJoCo (Wang et al., 2026) studies 11 dexterous manipulation tasks based on MuJoCo simulation (Todorov et al., 2012). These efforts motivate benchmark designs that evaluate more dexterous manipulation tasks and consider the ability to compose skills.

Despite this progress, existing benchmarks provide different combinations of task diversity, horizon, embodiment coverage, and evaluation granularity. DexArena complements these efforts by providing a unified benchmark for long-horizon dexterous manipulation across heterogeneous end-effectors. It features explicitly staged tasks with physically grounded success conditions and supports evaluation of both intermediate progress and full-task completion. By combining multi-stage task composition and embodiment variation within a common simulation framework, DexArena enables systematic evaluation of the capabilities required for complex dexterous manipulation.

Due to the page limit, additional related work on policy learning is included in Section A.1.

3 DexArena Benchmark

DexArena is a physics-based simulation benchmark for long-horizon dexterous manipulation built on the SAPIEN simulator (Xiang et al., 2020). This simulator allows us to run multiple environments in parallel for fast inference while maintaining high fidelity scene rendering. DexArena comprises **10 multi-stage manipulation tasks and 30 stage-level tasks**, spanning scenarios from household activities to industrial-style manipulation, with both single-arm and bimanual settings. Each multi-stage task is decomposed into explicitly defined stages with physically grounded success conditions,

Table 1: **Capability comparison with related benchmarks.** Green checks indicate emphasized capabilities. Red crosses indicate capabilities not emphasized.

Benchmark	Num. of tasks	Dexterous hands	Bimanual tasks	Cross-embodiment	Teleop demos	Long-horizon	Task composition
Adroit (Rajeswaran et al., 2018)	4	✓	×	×	100	×	×
RLBench (James et al., 2020)	100	×	×	×	×	×	×
Meta-World (Yu et al., 2020)	50	×	×	×	×	×	×
ManiSkill2 (Gu et al., 2023)	20	×	×	×	×	×	×
DexArt (Bao et al., 2023)	4	✓	×	×	×	×	×
RoboCasa (Nasiriany et al., 2024)	100	×	×	×	×	✓	✓
DexMimicGen (Jiang et al., 2025)	9	✓	✓	✓	60	✓	×
RoboTwin2.0 (Mu et al., 2025)	50	×	✓	✓	×	×	×
RoboDojo (Chen et al., 2026)	60	×	✓	✓	3500	✓	×
DexJoCo (Wang et al., 2026)	11	✓	✓	✓	1100	✓	×
DexArena (ours)	10+30	✓	✓	✓	3,000	✓	✓

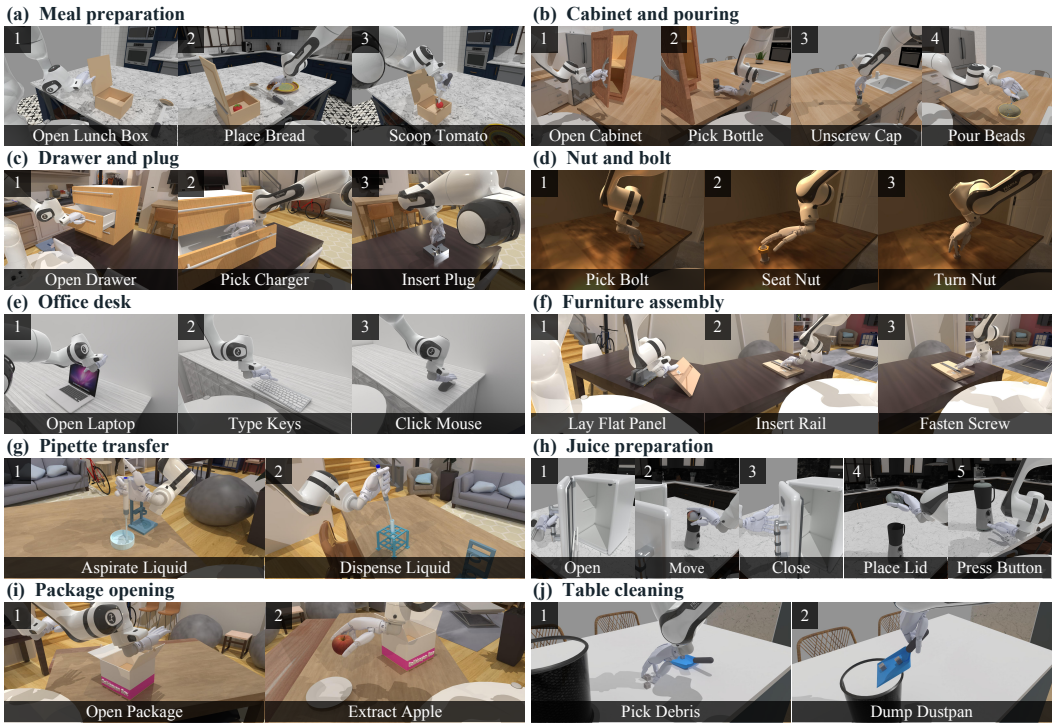


Figure 2: **Long-horizon tasks and their stages.**

enabling a policy to be evaluated both on individual skills and on their composition into a complete task. The benchmark supports four end-effector types: a parallel-jaw gripper and three dexterous hands (Sharpa Hand, Wuji Hand, and XHand). By combining long-horizon task composition, contact-rich dexterous interaction, multiple embodiments, and standardized evaluation, DexArena provides a unified testbed for studying the capabilities and limitations of modern robot learning methods.

3.1 Task Design

Every task in DexArena is built on the same workstation, the same physics and the same parameter layers. Only the objects, the layout and the success conditions are task-specific. This matches each stage-level task with the corresponding long-horizon stage, and keeps the benchmark easy to extend.

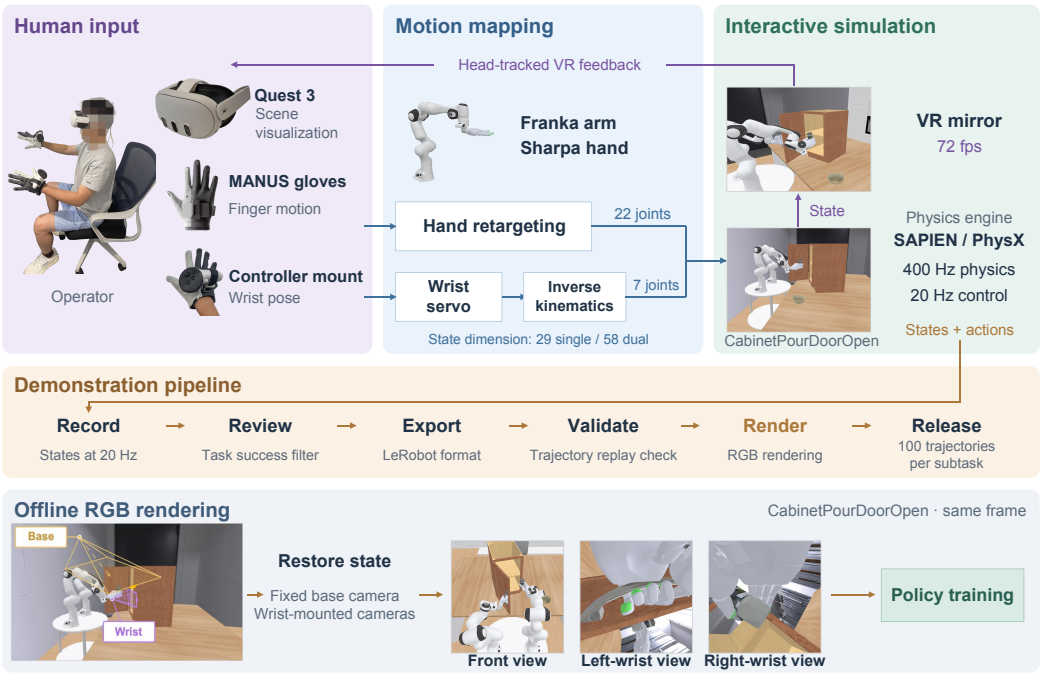


Figure 3: **Data collection pipeline.** The operator wears a VR headset and gloves, with a controller mounted on the back of each hand. Finger and wrist inputs are mapped to absolute joint targets. The scene is rendered in VR providing a better view for the operator. Each episode is recorded, then reviewed, exported, validated and rendered offline.

Tasks and their decomposition. Each long-horizon task is decomposed into two to five stages, as shown in Figure 2. The resulting 30 stages fall into six motion categories: *Opening and Closing*, *Grasping and Placement*, *Insertion and Assembly*, *Twisting and Screwing*, *Pressing*, and *Scooping and Pouring*. The long-horizon task and its stages share the same layout, randomization ranges, and stage-level success conditions. The workstation and the initial configuration of the arm are also shared. For each stage-level task, only the arm(s) and hand(s) required by that task are retained in the scene. All these settings allow a long-horizon policy to be evaluated stage by stage.

Success conditions. Every stage terminates when thresholds over physical quantities are satisfied, such as joint angles of articulated parts, relative poses, and insertion depths. Conditions rely on live object poses, so they stay valid under randomization. Most conditions must hold for 3 to 40 consecutive steps depending on the stage, so that an unstable contact is not mistaken for completion.

Layered parameterization. Environment parameters are organized into four explicit layers. The environment, hand and arm layers are fixed across all tasks. Adding a task means updating only its task layer, without modifying the other three. This allows a stage-level policy to be deployed unchanged inside the long-horizon task, and makes new tasks easy to build.

Scenes, randomization and rendering. Initial object and robot poses are randomized at the beginning of each episode depending on the random seed. Given the saved scene state of every frame, a trajectory can be re-rendered with different cameras while preserving the robot and object poses. A stage-level task can also be evaluated inside a realistic indoor scene while preserving its layout and physics.

The details about task decomposition and parameterization are included in Sections A.2 and A.3.

3.2 Teleoperation Data Collection

Teleoperation. The operator wears a Quest 3 headset and a pair of MANUS Meta Gloves Pro, with a Quest 3 controller mounted on the back of each hand for the wrist pose (Figure 3). Finger motion

is mapped to the 22 joints of the Sharpa hand by the official retargeting software (Sharpa Robotics, 2026), which uses the same kinematic model as our simulated hand. Other supported hands are retargeted with an open-source optimization-based library (Qin et al., 2023). The wrist pose is converted to arm joint targets by inverse kinematics, with joint speeds capped at certain limits. Recording runs at 20 Hz and stops automatically one second after the task succeeds. The action space is 29/58-dimensional for single/bimanual tasks, corresponding to absolute joint targets of the single arm/dual arms (7/14) and the single hand/dual hands (22/44). Each frame stores the observation, the action, a sparse reward and episode flags in the LeRobot (Cadene et al., 2024) format; the scene state of every frame is kept alongside for restoration. More details about teleoperation are in Appendix A.4.

Validation and rendering. Episodes are reviewed after each session and only successful ones are kept. Each dataset then goes through two checks. First, recorded frames are restored in a new simulator instance and must reproduce the recorded scene state and rendered images, which makes offline rendering reliable. Camera views are rendered this way from the restored scene state: one fixed scene camera and one wrist camera per arm in use, each 256×256. Camera placement and rendering settings are given in Appendix A.5. Second, the recorded actions are replayed open-loop from the recorded starting state. In contact-rich phases such as grasping, insertion or pouring, tiny differences in the contact computation of the simulator can accumulate over time and may lead to failure during replay. For each of the 30 stage-level tasks, we release 100 demonstrations that succeed on replay, yielding 3000 episodes and 9.1 hours in total, selected from 4112 accepted episodes. More details about data post-processing are in Appendix A.5.

3.3 Policy Learning

Here, we briefly introduce how we adapt all the baseline methods for this benchmark. Implementation details and training configurations for each method are included in Section A.6. The baseline methods can be mainly categorized into three types: Demonstration-guided Reinforcement Learning (RL), Imitation Learning (IL), and Vision-Language-Action (VLA) models. All methods take image observations and robot proprioception as inputs. For single-arm/bimanual tasks, visual observations consist of images from the front camera and the corresponding wrist camera(s). Across all policy families, actions are represented as joint-space commands.

Demo-guided RL To avoid task-specific reward engineering, all DexArena tasks use sparse rewards defined directly by their success conditions. However, training image-based RL policies from sparse rewards alone is challenging, particularly for dexterous manipulation. We therefore incorporate human demonstrations into RL training. For PPO (Schulman et al., 2017), we use three mechanisms to leverage the demonstrations. First, we perform a Behavior Cloning (BC) warm-up and retain a BC loss term during subsequent RL training. Second, we reset the environment from physical states sampled from demonstration trajectories with a certain probability, allowing the policy to progressively learn from states closer to successful behaviors. Third, we initialize the action distribution of the policy and constrain its output range using the action statistics of the demonstrations. We also evaluate Reinforcement Learning with Prior Data (RLPD) (Ball et al., 2023), a demonstration-guided RL method that combines offline demonstrations with online experience throughout training. Specifically, each training batch consists of an equal number of samples from the demonstration dataset and the online replay buffer.

Imitation Learning To adapt Action Chunking with Transformers (ACT) (Zhao et al., 2023) to dexterous manipulation, we extend its state and action spaces to include all arm and hand joints. To prevent the high-dimensional hand action from dominating optimization, we compute reconstruction losses separately for the arm and hand groups and assign a larger weight to the arm group. We additionally condition the policy on multi-view temporal observations. We adapt Diffusion Policy (DP) (Chi et al., 2023) to the same dexterous-hand control interface by jointly generating absolute target positions for all arm and hand joints. The policy is conditioned on the current robot joint positions and a short history of image observations.

VLA We adapt GR00T (NVIDIA et al., 2025) to single- and bimanual dexterous manipulation by introducing new embodiment-specific state and action encoders, while retaining the original action-head dimensionality. To account for the distinct characteristics of arm and hand control, we nor-

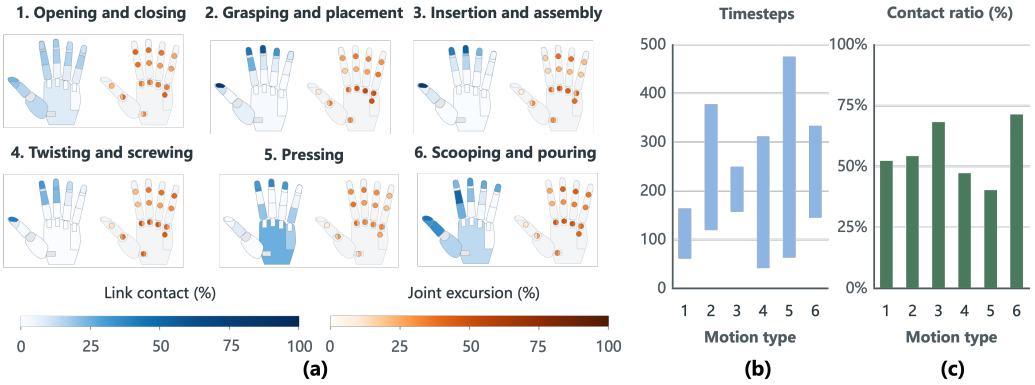


Figure 4: **Dataset Statistics Visualization**. Link contact: fraction of hand-contact frames. Joint excursion: motion amplitude of each hand joint averaged over episodes. Two degree-of-freedom joints are drawn as two half markers, the left half for bending and the right half for spreading. Timesteps: range of median episode lengths across tasks in each motion category. Contact ratio: fraction of frames with hand contact (any finger or the palm touching the primary object).

malize their joint spaces separately and use a mixed action representation, with relative joint offsets for the arms and absolute joint targets for the fingers. All model parameters, including the visual encoder and language backbone, are fine-tuned on the DexArena demonstrations.

4 Experiments

As introduced in Section 3.3, we evaluate five policy learning baselines along three axes: individual stage performance, generalization to varying scenarios, and long-horizon performance. All models are trained using the teleoperation dataset from Section 3.2. For each stage-level task or long-horizon task, we evaluate the policy for 50 episodes and report the percentage of successful evaluation episodes. The episode horizon, initial configuration randomization and embodiment settings are set up according to Section 3.1 and are matched across baselines.

4.1 Teleoperation Dataset

The released dataset contains 100 demonstrations for each of the 30 stage-level tasks, 3000 episodes and 9.1 hours in total. We analyze it from three angles: hand usage, diversity of episodes and open-loop replay. Figure 4 shows that the six motion categories differ in hand-link contact, joint motion, episode length and contact ratio. Palm trajectories show substantial motion and inter-episode diversity. The mean path length ranges from 0.35 to 2.04,m, while the mean pairwise distance between palm trajectories ranges from 3.3 to 10.2,cm across stage-level tasks. Open-loop replay succeeds in 3222 of the 4112 accepted episodes, and tends to be less successful for more contact-rich tasks, from 98% for opening and closing to 65% for insertion and assembly. Definitions and per stage-level task results are given in Section A.7.

4.2 Stage-level Task Performance

We first train and evaluate each baseline on all 30 stage-level tasks individually. Each episode starts from a valid initial state for the corresponding stage, and success is determined by its physically grounded completion criterion. Table 2 reports the success rate for every policy on each stage.

4.3 Generalization Performance in Varying Visual Scenes

We next evaluate the same trained policies on the same 30 stage-level tasks under three different scenarios, as shown in Section A.8, without additional training or adaptation. For each stage and policy, Table 3 reports the average success rate across the evaluation scenarios. The final row shows

Table 2: **Stage-level task performance.** Success rates (%, ↑) on all 30 stage-level tasks in the training scene shown in Figure 6, grouped by their long-horizon task, with 50 evaluation episodes conducted per task.

Long-horizon	Stages	Demo-guided RL		Imitation learning		
		PPO	RLPD	ACT	DP	GR00T
Meal preparation	OpenLunchBox	74	100	100	96	100
	PlaceBread	12	86	88	20	60
	ScoopTomato	0	0	44	0	12
Cabinet and pouring	OpenCabinet	62	100	94	88	92
	PickBottle	0	14	68	18	78
	UnscrewCap	0	2	62	50	62
	PourBeads	0	96	100	32	78
Drawer and plug	OpenDrawer	98	100	98	98	98
	PickCharger	0	62	92	0	78
	InsertPlug	0	0	16	12	6
Nut and bolt	PickBolt	0	28	8	22	44
	SeatNut	0	34	12	6	26
	TurnNut	76	100	96	100	100
Office desk	OpenLaptop	100	100	100	88	100
	TypeKeys	0	0	0	0	32
	ClickMouse	96	100	14	92	98
Furniture assembly	LayFlatPanel	0	96	78	24	64
	InsertRail	0	0	22	2	16
	FastenScrew	100	100	0	0	84
Pipette transfer	AspirateLiquid	0	0	40	2	8
	DispenseLiquid	0	0	0	2	8
Juice preparation	OpenFridge	90	98	100	88	94
	AppleToBlender	0	0	92	42	54
	CloseFridge	100	100	100	100	98
	PlaceLid	0	78	26	26	42
	PressButton	26	100	66	72	64
Package opening	OpenPackage	2	92	44	2	64
	ExtractApple	0	42	88	22	100
Table cleaning	PickDebris	0	0	0	2	0
	DumpDustpan	0	0	92	70	82
Average		27.9	54.3	58.0	39.2	61.4

average scores over all 30 stage-level tasks. Comparison with Table 2 quantifies the change in performance under scenario variation.

4.4 Long-Horizon Performance

Finally, we evaluate all five baselines on the 10 long-horizon tasks. For the single-task policies, we sequentially execute the learned stage-level policies and switch policies when the success condition of the current stage is met. For the VLA baseline, we train one model for each long-horizon task using demonstrations from all of its constituent stage-level tasks and switch the instruction between stages. The environment is initialized only at the beginning of the full task and is not reset between stages. An episode succeeds only if all required stages are completed in order and the final success condition of the full task is satisfied within the evaluation horizon. Unfortunately, almost all policies failed in the long-horizon tasks. We observe that although individual stage-level policies can achieve non-trivial success rates, composing stage-level behaviors does not reliably yield successful long-horizon execution. Per-task results and further evaluation details are provided in Appendix A.9 (Table 16).

Table 3: **Generalization performance.** Success rates (%, ↑) on the same 30 stage-level tasks under varying scenarios, grouped by their long-horizon task. Results are reported as mean$_{\pm \text{std}}$ across the three held-out scenes, with 50 evaluation episodes conducted per scene.

Long-horizon	Stages	Demo-guided RL		Imitation learning		
		PPO	RLPD	ACT	DP	GR00T
Meal preparation	OpenLunchBox	$35.3_{\pm 14.5}$	$33.3_{\pm 57.7}$	$100.0_{\pm 0.0}$	$35.3_{\pm 31.6}$	$100.0_{\pm 0.0}$
	PlaceBread	$10.0_{\pm 3.5}$	$0.0_{\pm 0.0}$	$40.0_{\pm 30.0}$	$8.7_{\pm 4.6}$	$24.0_{\pm 26.2}$
	ScoopTomato	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$23.3_{\pm 9.0}$	$0.0_{\pm 0.0}$	$6.7_{\pm 6.1}$
Cabinet and pouring	OpenCabinet	$8.7_{\pm 9.0}$	$63.3_{\pm 54.9}$	$93.3_{\pm 6.4}$	$84.0_{\pm 11.1}$	$78.7_{\pm 17.9}$
	PickBottle	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$76.0_{\pm 8.0}$	$4.0_{\pm 5.3}$	$46.0_{\pm 36.7}$
	UnscrewCap	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$4.0_{\pm 3.5}$	$1.3_{\pm 2.3}$	$9.3_{\pm 14.5}$
	PourBeads	$0.7_{\pm 1.2}$	$0.0_{\pm 0.0}$	$42.0_{\pm 22.3}$	$6.0_{\pm 10.4}$	$16.7_{\pm 17.5}$
Drawer and plug	OpenDrawer	$97.3_{\pm 4.6}$	$95.3_{\pm 4.2}$	$82.7_{\pm 10.1}$	$4.0_{\pm 5.3}$	$90.0_{\pm 10.6}$
	PickCharger	$0.0_{\pm 0.0}$	$5.3_{\pm 9.2}$	$88.0_{\pm 5.3}$	$0.0_{\pm 0.0}$	$64.7_{\pm 19.6}$
	InsertPlug	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.7_{\pm 1.2}$	$0.0_{\pm 0.0}$	$2.0_{\pm 3.5}$
Nut and bolt	PickBolt	$0.7_{\pm 1.2}$	$0.0_{\pm 0.0}$	$8.7_{\pm 5.0}$	$2.0_{\pm 3.5}$	$21.3_{\pm 13.3}$
	SeatNut	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$2.7_{\pm 2.3}$	$0.7_{\pm 1.2}$	$2.0_{\pm 3.5}$
	TurnNut	$26.7_{\pm 11.0}$	$11.3_{\pm 16.3}$	$44.0_{\pm 19.7}$	$11.3_{\pm 12.7}$	$80.0_{\pm 19.7}$
Office desk	OpenLaptop	$99.3_{\pm 1.2}$	$48.0_{\pm 50.1}$	$84.7_{\pm 18.1}$	$88.7_{\pm 3.1}$	$95.3_{\pm 8.1}$
	TypeKeys	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$17.3_{\pm 17.0}$
	ClickMouse	$98.0_{\pm 3.5}$	$6.7_{\pm 11.5}$	$12.7_{\pm 4.6}$	$68.7_{\pm 15.0}$	$100.0_{\pm 0.0}$
Furniture assembly	LayFlatPanel	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$54.0_{\pm 27.5}$	$0.0_{\pm 0.0}$	$34.7_{\pm 28.4}$
	InsertRail	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$6.0_{\pm 10.4}$
	FastenScrew	$86.0_{\pm 24.2}$	$70.7_{\pm 50.8}$	$85.3_{\pm 11.5}$	$0.0_{\pm 0.0}$	$20.0_{\pm 26.2}$
Pipette transfer	AspirateLiquid	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$8.7_{\pm 8.1}$	$1.3_{\pm 1.2}$	$2.0_{\pm 2.0}$
	DispenseLiquid	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$2.0_{\pm 2.0}$
Juice preparation	OpenFridge	$65.3_{\pm 22.0}$	$78.7_{\pm 35.2}$	$92.7_{\pm 9.5}$	$68.0_{\pm 33.3}$	$70.0_{\pm 45.1}$
	AppleToBlender	$0.7_{\pm 1.2}$	$0.0_{\pm 0.0}$	$71.3_{\pm 14.2}$	$7.3_{\pm 12.7}$	$18.0_{\pm 2.0}$
	CloseFridge	$100.0_{\pm 0.0}$	$100.0_{\pm 0.0}$	$99.3_{\pm 1.2}$	$66.0_{\pm 47.3}$	$92.0_{\pm 8.7}$
	PlaceLid	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$18.0_{\pm 11.1}$	$1.3_{\pm 2.3}$	$28.7_{\pm 12.9}$
	PressButton	$18.0_{\pm 5.3}$	$18.7_{\pm 32.3}$	$54.0_{\pm 19.3}$	$52.7_{\pm 40.6}$	$42.7_{\pm 10.1}$
Package opening	OpenPackage	$2.0_{\pm 0.0}$	$2.7_{\pm 4.6}$	$36.7_{\pm 22.1}$	$1.3_{\pm 2.3}$	$35.3_{\pm 22.1}$
	ExtractApple	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$78.7_{\pm 3.1}$	$3.3_{\pm 3.1}$	$54.7_{\pm 39.5}$
Table cleaning	PickDebris	$1.3_{\pm 2.3}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$1.3_{\pm 1.2}$
	DumpDustpan	$0.7_{\pm 1.2}$	$0.0_{\pm 0.0}$	$59.3_{\pm 17.9}$	$1.3_{\pm 2.3}$	$88.7_{\pm 2.3}$
Average		$\mathbf{21.7_{\pm 3.5}}$	$\mathbf{17.8_{\pm 10.9}}$	$\mathbf{45.4_{\pm 3.6}}$	$\mathbf{17.2_{\pm 5.7}}$	$\mathbf{41.7_{\pm 14.2}}$

5 Conclusion

We introduced DexArena, a benchmark for studying long-horizon dexterous manipulation through 10 multi-stage tasks and 30 physically grounded stage-level tasks. The shared task and stage definitions, four end-effectors, replayable simulation, and teleoperated demonstrations support evaluation of individual skills, scene variation, and sequential task completion in a common framework. The baseline results show that performance on isolated stage-level tasks does not by itself yield reliable long-horizon behavior. Performance also varies substantially across stages and evaluation scenes, exposing persistent challenges in contact-rich manipulation and robustness to visual variation.

6 Limitation

Although we calibrate the simulator parameters to better match real-world robot dynamics, accurately reproducing the complex contact dynamics of dexterous manipulation remains challenging. In particular, contacts between multiple fingers, objects, and articulated components can be sensitive to small discrepancies in geometry, friction, and actuation, leaving a non-negligible sim-to-real gap. Further reducing this gap will require more extensive system identification and validation against physical-robot interactions.

AI USE STATEMENT

We used generative AI tools to assist with implementing the simulation tasks, reviewing experimental methodology and results, and polishing the language of the manuscript. All AI-assisted code, analyses, and text were reviewed, and the simulation implementation was tested by the authors. We take responsibility for the final content of this work, including all text, claims, code, data, and other artifacts.

ETHICS STATEMENT

We have considered the issues covered by the ICLR Code of Ethics and do not identify any ethical concerns raised by this work.

REPRODUCIBILITY STATEMENT

We will release the complete source code, datasets, training and evaluation scripts, and trained model checkpoints. The main paper and appendix document the task construction, teleoperation data collection and validation, policy implementations, training configurations, and evaluation protocols (see Sections 3.2 and 4, and Appendices A.5 and A.6). Moreover, we configure the random seeds explicitly for both training and evaluation and include them in the released configurations so that the reported results can be reproduced.

REFERENCES

Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 1577–1594. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/ball23a.html.

Chen Bao, Helin Xu, Yuzhe Qin, and Xiaolong Wang. Dexart: Benchmarking generalizable dexterous manipulation with articulated objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 21190–21200, June 2023.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*, LosAngeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.010.

Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. https://github.com/huggingface/lerobot, 2024.

Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Weijie Wan, Baijun Chen, Haoran Lu, Haowen Yan, Honghao Su, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

Jiayuan Gu et al. Maniskill2: A unified benchmark for generalizable manipulation skills. In *ICLR*, 2023.

Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J. Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 2020.

Zhenyu Jiang, Yuqi Xie, Kevin Lin, Zhenjia Xu, Weikang Wan, Ajay Mandlekar, Linxi Fan, and Yuke Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

Yao Mu, Tianxing Chen, Zanxin Chen, Shijia Peng, Zhiqian Lan, Zeyu Gao, Zhixuan Liang, Qiaojun Yu, Yude Zou, Mingkun Xu, Lunkai Lin, Zhiqiang Xie, Mingyu Ding, and Ping Luo. Robotwin: Dual-arm robot benchmark with generative digital twins. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 27649–27660, June 2025.

Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. In *ArXiv Preprint*, March 2025.

Yuzhe Qin, Wei Yang, Binghao Huang, Karl Van Wyk, Hao Su, Xiaolong Wang, Yu-Wei Chao, and Dieter Fox. Anyteleop: A general vision-based dexterous robot arm-hand teleoperation system. In *Robotics: Science and Systems*, 2023.

Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018. doi: 10.15607/RSS.2018.XIV.049.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Sharpa Robotics. Manus SDK Integration for SharpaWave. https://github.com/sharpa-robotics/sharpa-manus-sdk, 2026. Commit b87bff5, accessed 2026-09-22.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.

Hanwen Wang, Weizhi Zhao, Xiangyu Wang, Siyuan Huang, He Lin, Boyuan Zheng, Rongtao Xu, Gang Wang, Yao Mu, He Wang, Lue Fan, Hongsheng Li, Zhaoxiang Zhang, and Tieniu Tan. Dexjoco: A benchmark and toolkit for task-oriented dexterous manipulation on mujoco, 2026. URL https://arxiv.org/abs/2605.16257.

Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. Sapien: A simulated part-based interactive environment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World action models are zero-shot policies, 2026. URL https://arxiv.org/abs/2602.15922.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In Leslie Pack Kaelbling, Danica Kragic, and Komei Sugiura (eds.), *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pp. 1094–1100. PMLR, 30 Oct–01 Nov 2020. URL https://proceedings.mlr.press/v100/yu20a.html.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. doi: 10.15607/RSS.2023.XIX.016.

A Appendix

A.1 Related Work on Policy Learning

Data-driven policy learning has become a dominant paradigm for robot manipulation, with imitation learning (IL) and reinforcement learning (RL) providing complementary approaches to acquiring manipulation skills. Early data-driven methods primarily focused on learning single-task policies from demonstrations. Action Chunking with Transformers (ACT) (Zhao et al., 2023) predicts temporally coherent action sequences from demonstrations, while Diffusion Policy (DP) (Chi et al., 2023) models multi-modal action distributions through iterative denoising. These methods have demonstrated strong performance in individual manipulation tasks, but their training and generalization are typically constrained by the scale and diversity of the collected demonstrations.

Recent research has increasingly moved toward multi-task and large-scale pretrained policies, enabled by the availability of diverse robot datasets and large pretrained models. Vision-Language-Action (VLA) models incorporate visual and language knowledge to learn general-purpose manipulation behaviors across tasks and embodiments. For example, π_0 (Black et al., 2025) builds a flow-based action expert based pre-trained Vision-Language Model (VLM) for general robot control, while GR00T (NVIDIA et al., 2025) develops a hierarchical foundation model for generalist robot manipulation. Beyond VLA policies, emerging approaches such as World-Action-Model (Ye et al., 2026) further incorporate video-generative models to learn predictive representations of the environment and support video-pre-training and policy learning in a unified model architecture.

In parallel, reinforcement learning provides an important mechanism for learning and adapting policies through environment interaction, particularly in simulation where large amounts of interaction data can be collected efficiently. PPO employs clipped on-policy optimization for policy improvement (Schulman et al., 2017), while RLPD combines demonstrations with online experience through replay-based learning (Ball et al., 2023). RL can therefore serve both as a standalone policy-learning paradigm and as a complementary approach for post-training pretrained or imitation-learned policies, allowing them to further optimize task-specific objectives through interaction.

These developments motivate evaluating manipulation policies across different levels of data scale, prior knowledge, and interaction. DexArena provides a common evaluation framework for comparing such data-driven approaches under long-horizon dexterous manipulation, where performance depends not only on learning individual skills but also on composing them over multiple stages and handling diverse embodiments and interaction dynamics.

A.2 Task Information

Table 4 groups the 30 stage-level tasks by the dominant motion they require.

Table 5 lists every stage-level task with the hand it uses, its episode horizon, and the condition under which it is credited as complete.

Table 4: The 30 stage-level tasks by motion category, with properties of the released demonstrations (100 per stage-level task). Time to success: range of the stage-level task medians. Contact is estimated by re-simulating each recorded frame (threshold 0.1 N). Contact ratio: fraction of frames, up to first success, in which a finger or the palm touches the primary object of the stage-level task. Fingers: number of distinct fingers in contact, averaged over hand-contact frames. Stage-level task values pool the frames of its 100 episodes; category values weight stage-level tasks equally.

Category	#	Stages		Time (s)	Contact ratio	Fingers

Opening and closing	7	`OpenCabinet`
		`OpenFridge`
		`OpenLunchBox`
		`OpenPackage`
			`OpenDrawer`
			`CloseFridge`
			`OpenLaptop`
				3.0–8.1	0.52	2.3

Grasping and placement	8	`PickBottle`
		`PickBolt`
		`ExtractApple`
		`LayFlatPanel`
			`PickCharger`
			`PlaceBread`
			`AppleToBlender`
			`PickDebris`
				5.9–18.8	0.54	3.1

Insertion and assembly	4	`InsertPlug` `SeatNut`	`InsertRail` `PlaceLid`	7.8–12.4	0.68	2.7
Twisting and screwing	3	`UnscrewCap` `FastenScrew`	`TurnNut`	2.0–15.5	0.47	2.1

Pressing	5	`PressButton`
		`TypeKeys`
		`DispenseLiquid`
			`ClickMouse`
			`AspirateLiquid`
				3.1–23.7	0.40	1.9

Scooping and pouring	3	`ScoopTomato` `DumpDustpan`	`PourBeads`	7.2–16.6	0.71	3.1

Table 5: The 30 stage-level tasks: hand used, episode horizon in control steps, description, and success condition. "Hold n" means the condition must stay true for n consecutive control steps.

Stages	Hand	Horizon	Description	Success condition
Opening and closing				
OpenLunchBox	L	300	Open the lid of a lunch box.	Lid angle $\geq 90^\circ$, opened by the left hand; hold 20.
OpenCabinet	L	600	Open a cabinet door.	Door angle $\geq 80^\circ$; hold 5.
OpenDrawer	L	600	Pull a drawer open.	Drawer travel > 0.18 m; hold 5.
OpenFridge	L	650	Open a mini-fridge door.	Door angle $\geq 75^\circ$ and door speed ≤ 0.12 rad/s; hold 6.
CloseFridge	L	650	Close the fridge door from 75°.	Door angle $\leq 1^\circ$ and speed ≤ 0.12 rad/s; hold 6.
OpenLaptop	R	600	Open a laptop lid from 40–50°.	Lid reached $\geq 90^\circ$, then stays $\geq 85^\circ$; hold 4.
OpenPackage	B	500	Fold both flaps of a package open.	Both flaps rotated $> 90^\circ$ from closed.
Grasping and placement				
PickBottle	R	900	Take a bottle out of the cabinet and set it on a target.	Bottle out of the cabinet, upright within 0.15 m of the target, at rest, released; hold 5.
PickCharger	R	900	Lift a charger out of the drawer and place it.	Charger out of the drawer, within 0.14 m of the target, flat, at rest; hold 5.
PickBolt	L	700	Stand a bolt upright on its target.	Bolt head within 0.10 m of the target, tilt $\leq 5^\circ$, at rest, released; hold 3.
PlaceBread	R	400	Move the bread onto the plate.	Bread within 0.13 m of the plate centre and 0–7 cm above it; hold 20.

continued on next page

Table 5, continued

Stages	Hand	Horizon	Description	Success condition
ExtractApple	R	700	Take the apple out of the package and place it on the plate.	Apple lifted clear of the package, then on the plate within 5 cm, at rest, hand > 0.10 m away; hold 40.
AppleToBlender	R	700	Take the apple from the fridge into the blender pitcher.	Apple inside the pitcher and at rest, after leaving the fridge and being lifted ≥ 8.5 cm; hold 12.
LayFlatPanel	L	600	Take a panel from its rack and lay it flat in the placement zone.	Panel flat (normal $z \geq 0.95$) inside the zone, yaw $\leq 20^\circ$, at rest, released; hold 5.
PickDebris	R	900	Put three debris pieces into the dustpan.	All three pieces at rest inside the dustpan and none held; hold 8.
Insertion and assembly				
InsertPlug	R	900	Insert the charger plug into the socket.	Both pins 18–31 mm deep, lateral offset < 9 mm, tilt $\leq 18^\circ$, at rest; hold 5.
InsertRail	R	700	Slide a rail into the groove of the panel.	Rail seated: depth error ≤ 2 mm, lateral ≤ 4 mm, aligned, released, at rest; hold 5.
SeatNut	R	800	Place the nut onto the bolt.	Nut on the bolt axis (xy < 5 mm), at seat height within 3 mm, tilt $\leq 12^\circ$; hold 3.
PlaceLid	R	400	Put the lid on the blender pitcher.	Lid lifted ≥ 3 cm once, then seated: xy ≤ 5 mm, z ≤ 8 mm, tilt $\leq 15^\circ$; hold 15.
Twisting and screwing				
UnscrewCap	R	1400	Unscrew the bottle cap and set it down.	Cap rotated $\geq 180^\circ$ and detached, then flat within 0.10 m of the target, at rest, hand ≥ 65 mm away; hold 5.
TurnNut	R	900	Tighten the seated nut on the bolt.	Net nut rotation $\geq 180^\circ$ while seated; hold 5.
FastenScrew	R	600	Turn a wing screw on the rail to its stop.	Screw angle $\geq 87^\circ$.
Pressing				
PressButton	R	400	Press the blender's power button.	Button pressed $\geq 80\%$ of its 2 mm travel; hold 4.
ClickMouse	R	500	Click the left mouse button.	Left button $\geq 60\%$ travel with a finger pad over it, right button $< 35\%$; hold 3.
TypeKeys	L	700	Type a fixed five-key sequence ending with Enter.	Each key in turn pressed $\geq 60\%$ with a pad on the keycap, held 4 steps, then released.
AspirateLiquid	R	800	Draw liquid from the sample cup with the pipette.	Plunger pressed ≥ 7.5 mm with the tip in the cup, then released; pipette back in its stand; hand away.
DispenseLiquid	L	1100	Dispense from the pipette into the test tube.	Plunger pressed ≥ 7.5 mm with the tip in the tube, then released; pipette back in its stand; hand away.
Scooping and pouring				
ScoopTomato	R	1200	Scoop the tomato with the spoon onto the plate.	Tomato carried on the spoon, then on the plate within 9.4 cm and 0–6 cm above it; hold 20.
PourBeads	R	700	Pour the beads from the bottle into the bowl.	≥ 12 of 16 beads in the bowl, ≤ 2 spilled, none airborne; hold 10.
DumpDustpan	R	600	Empty the dustpan into the bin.	All three debris pieces inside the bin; hold 10.

Table 6: The four parameter layers of a DexArena environment. The first three are shared by every task in the benchmark. The fourth belongs to the task designer.

Layer	Scope	Contents
Environment	benchmark-wide	Physics rate 400 Hz and control rate 20 Hz. Solver iterations 24/8, contact offset 1 mm, deterministic solver. Work table geometry and friction.
Hand	benchmark-wide	Joint stiffness, armature and joint friction from the vendor model, unscaled; damping from the model's damping ratio; torque limits from the URDF. Finger-pad contact model ($\mu = 1.2$, 5 mm torsional patch radius).
Arm	benchmark-wide	Absolute position of the platform and of the arm roots relative to the work table. Arm start configuration.
Task	per task	Objects, their contact materials, masses and inertias. Layout and its randomization ranges. Success conditions and hold durations. Per-episode initialization.

Table 7: Values of the three fixed layers. Hand values are for the Sharpa Hand.

Environment layer	
Rates	Physics 400 Hz, control 20 Hz (20 physics sub-steps per control step)
Solver	TGS, 24 position and 8 velocity iterations, PCM, friction every iteration
Contact	Contact offset 1 mm, rest offset 0, bounce threshold 2 m/s
Integration	Enhanced determinism on, sleeping disabled, CCD off, gravity 9.81 m/s^2
Materials	Fallback $\mu = 0.30$; table and ground $\mu = 0.30$; restitution 0 throughout
Hand layer (Sharpa Hand, 22 joints)	
Joint drives	Stiffness 0.9–13.2 N m/rad, armature and Coulomb joint friction as in the vendor model, unscaled; damping 0.02–0.37 N m s/rad from the model's damping ratio of 0.9
Torque limits	0.19–3.3 N m per joint, from the URDF effort limits
Finger pads	Pads and distal phalanges: $\mu = 1.2$, restitution 0, torsional patch radius 5 mm (2 mm minimum)
Other hand links	$\mu = 0.35$; self-collision enabled
Mount	90° adapter between the arm flange and the hand
Arm layer (Franka Panda)	
Placement	Platform root at $x = -0.870$ m, work-table centre at $x = +0.180$ m, arm roots at $y = \pm 0.220$ m (bimanual), all in world coordinates
Start configuration	$(-0.50, -0.20, 0.00, -2.60, 0.00, 2.40, 0.79)$ rad, plus per-episode noise from the task layer
Control	Joint impedance: stiffness 1000 N m/rad, damping 100 N m s/rad, torque limit 100 N m, gravity compensation on; actions are absolute joint targets
Materials	Arm links $\mu = 0.30$, restitution 0

A.3 Parameters of the Environment

Environment parameters are organized into four explicit layers, as shown in Table 6.

Table 7 gives the values the first three layers are fixed to. The hand layer is shown for the Sharpa Hand, the hand used for all released data; the other supported hands follow the same layer with their own vendor values.

The task layer is the only layer a new task writes. It has to specify:

- the objects, with explicit mass and inertia consistent with the collision geometry; manipulated objects carry a contact material chosen by material class from a fixed table, fixtures use the default material; free bodies receive a velocity ceiling of 3 m/s and a depenetration ceiling of 0.5 m/s;
- the layout: one set of nominal poses shared by the long-horizon task and its stages, within 0.54–0.90 m of the arm base, and the per-episode randomization ranges, drawn from the episode's random stream together with the arm-start noise ($\sigma = 0.02$ rad per joint);

- the success conditions of each stage with their hold durations, the episode horizon, and the stage flags kept in the scene state;
- optional on-table guidance cues for the operator, which are visual only.

A.4 TELEOPERATION SYSTEM

Hardware. The operator wears a Quest 3 headset and a pair of MANUS Meta Gloves Pro. A Quest 3 controller is mounted on the back of each hand on a custom bracket and supplies the wrist pose. The whole system, simulation included, runs on one workstation with a single RTX 5090.

Retargeting. The glove reports a 25-node hand skeleton. For the Sharpa Hand it is passed to the hand vendor's retargeting solver (Sharpa Robotics, 2026), which returns a target angle for each of the 22 finger joints; the simulated hand is the kinematic model the solver was built for, so the angles are applied by joint name without rescaling. The other supported hands are retargeted with the open-source `dex-retargeting` library (Qin et al., 2023), with one configuration per hand. Both paths match finger kinematics only.

Wrist and arm. The controller pose is passed through a critically damped second-order reference shaper and a position servo. The servo detects tracking dropouts, which the headset reports as a frozen position with the tracking flag still set, and holds the last valid target instead of following the frozen one. The wrist target is solved into seven absolute arm joint targets by differential inverse kinematics with a null-space term that pulls the redundant degree of freedom towards a comfort configuration; the usual end-effector delta interface carries no such term, and over a long session the redundancy winds up against the joint limits. Per-step joint increments are clamped to the real Franka velocity limits, so every recorded action is kinematically executable on the real arm. Bimanual tasks run the same chain for each arm.

Rendering for the operator. The headset view is produced by a separate process that holds a render-only replica of the environment and is driven by the poses and joint positions published by the control process. It renders at 72 fps, the headset's frame rate, independently of the 20 Hz control loop.

Operator aids. The operator can freeze the fingers at their current targets while repositioning the wrist. On-table markers show where an object should be placed; they carry no collision geometry and are hidden in all released camera views.

Segmentation. An episode is armed by the operator and cut automatically 20 control steps (1 s) after the success condition first holds, so every demonstration ends in a settled state. Four operators collected the data over about two weeks.

A.5 DATA POST-PROCESSING, VALIDATION AND RELEASE

Review and export. Recorded episodes are staged per session and reviewed afterwards; only episodes that end in success are exported. The first three frames of each episode are dropped at export, as they precede the operator's first command. Each exported episode is a LeRobot episode with the per-frame columns of Table 8; the scene state of every frame is kept next to the dataset in the form the simulator restores from.

Table 8: Per-frame columns of a released episode.

Column	Content
`observation.state`	Robot joint positions and velocities followed by task-specific quantities (object poses, stage flags); dimension per stage in Table 9
`action`	Absolute joint targets, 7 arm + 22 hand per arm (29 / 58)
`next.reward`	1 while the success condition holds, else 0
`next.success`, `next.done`	Success and episode-end flags
`observation.environment_state`	Flattened poses and joint states of all scene bodies

Restoration check. For sampled episodes of every dataset, recorded frames are restored in a fresh simulator instance and compared with the recording: the scene state, the observation, the task's own

Table 9: Observation dimension of each stage. The action dimension is 29 for single-arm and 58 for bimanual stage-level tasks.

Stages	Obs	Stages	Obs	Stages	Obs
OpenLunchBox	85	PlaceBread	116	UnscrewCap	96
OpenCabinet	79	ExtractApple	85	TurnNut	97
OpenDrawer	79	AppleToBlender	88	FastenScrew	93
OpenFridge	74	LayFlatPanel	134	PressButton	111
CloseFridge	74	PickDebris	88	ClickMouse	149
OpenLaptop	123	InsertPlug	96	TypeKeys	390
OpenPackage	128	InsertRail	153	AspirateLiquid	83
PickBottle	97	SeatNut	95	DispenseLiquid	90
PickCharger	92	PlaceLid	121	ScoopTomato	118
PickBolt	82			PourBeads	92
				DumpDustpan	90

quantities and the rendered images must match. Quantities that depend on the solver's contact state cannot be reproduced from a restored frame and are excluded. A dataset is released only after this check passes.

Open-loop replay. The recorded actions of every accepted episode are replayed from the recorded initial state in a fresh simulator instance, and the episode counts as replayable if the success condition is reached. The solver's contact state cannot be restored, so a replay starts from a slightly different contact state even when the recorded scene state is restored. During contact-rich motion, such differences can grow, whereas a human operator would correct them in the loop.

Release. For each stage the released 100 demonstrations are the replayable episodes.

Camera views. Camera images are rendered offline by restoring each frame and rendering a fixed scene view and one wrist view per arm in use. The scene camera is placed 1.08 m above the table behind the workspace with its image plane tilted 20° from the tabletop, aimed at the centre of the task's scene content. Each wrist camera sits on the wrist link with calibrated extrinsics. All views are rendered at 256×256 with a 90° vertical field of view. Guidance markers are hidden. Videos are encoded with AV1, and each released video is decoded and compared with the rendered frames (PSNR $\geq$ 30 dB) before release, so that the released video is a faithful copy of the rendering, with no dropped or misaligned frames.

A.6 Policy Implementations and Training Configurations

Proximal Policy Optimization (PPO). At step t the agent receives an observation o_t. It consists of two RGB images from fixed cameras and a proprioceptive vector p_t. p_t holds only quantities that are also available on the real robot: joint positions and velocities and the palm pose (TCP). Privileged simulator states are explicitly excluded. $n(\cdot)$ denotes standardization of p_t with running estimates of its mean and variance. Each image encoder uses one convolutional trunk per camera, followed by a linear projection, LayerNorm and $\tanh$; the per-camera outputs are concatenated into a latent vector. The actor and the critic own separate encoders f_a and f_v, each trained only by its own loss. The action $a_t \in \mathbb{R}^{|A|}$ is the vector of absolute joint targets of the arm–hand system ($|A| = 29$). The policy is a Gaussian with a bounded mean and a state-independent scale:

$$\pi_\psi(a_t \mid o_t) = \mathcal{N}\big(a_t;\ \mu_\psi(o_t),\ \mathrm{diag}\,\sigma^2\big), \qquad \mu_\psi(o_t) = b + c \odot \tanh\big(g_\psi([f_a(o_t),\ n(p_t)])\big). \quad (1)$$

g_ψ represents an MLP with parameters ψ. $\sigma \in \mathbb{R}^{|A|}$ represents a learned standard-deviation vector that does not depend on o_t. b and c are the per-joint center and half-width of the interval that bounds the mean. This interval is the demonstrations' action range, padded by a small margin and intersected with the joint limits.

The output bias of g_ψ is initialized so that the untrained policy outputs the mean demonstration action, and σ is initialized to a small value. The critic $V_\omega(o_t)$ is an MLP applied to $[f_v(o_t),\ n(p_t)]$. Let $\mathrm{succ}_t \in \{0, 1\}$ be the simulator's success flag. The reward is this flag itself, $r_t = \mathrm{succ}_t$. The episode terminates on success; reaching the time limit is a truncation, not a termination, so $V_\omega \in [0, 1]$. Advantages $\hat{A}_t$ are computed with generalized advantage estimation (discount γ, trace

parameter λ), and the value targets are $\hat{R}_t = \hat{A}_t + V_\omega(o_t)$. Bootstrapping happens only at truncations, using $V_\omega(o_t^{\text{fin}})$, where o_t^{fin} is the true final observation before the automatic reset. A successful step contributes exactly r_t.

Demonstrations $\mathcal{D}_{\text{demo}} = \{(o^d, a^d)\}$ carry no behavior log-probabilities, so they cannot enter the on-policy objective directly. They are used in three other ways. First, before any rollout the actor is warmed-up by minimizing the BC loss $\mathcal{L}_{\text{BC}} = \mathbb{E}_{\mathcal{D}_{\text{demo}}} \big\| \mu_\psi(\text{aug}(o^d)) - a^d \big\|_2^2$, where $\text{aug}(\cdot)$ is a random image shift. During this warm-up, the proprioceptive statistics are initialized from the demonstrations. Second, $\mathcal{L}_{\text{BC}}$ remains as an anchor during training. Third, environments are reset to demonstration states under a reverse curriculum. Let $q \in [0, 1]$ be the normalized progress of a frame within its demonstration. At each reset, with probability $p(\tau_c)$ the simulator is restored to the physical state of a demonstration frame with $q \geq 1 - f(\tau_c)$. Here τ_c is a curriculum clock that advances only while the recent training success rate exceeds a threshold. As τ_c advances, p decays and f grows linearly. Early episodes therefore start close to success, and later episodes increasingly start from the task's own initial-state distribution. Each update runs over mini-batches of the latest rollout and minimizes

$$\mathcal{L} = c_v \, \mathcal{L}_V + \kappa \big[\mathcal{L}_{\text{clip}} + \beta(t) \, \mathcal{L}_{\text{BC}} \big], \qquad \mathcal{L}_{\text{clip}} = \mathbb{E}\Big[\max\big(-\bar{A}\,\rho, \; -\bar{A}\,\text{clip}(\rho, 1-\epsilon, 1+\epsilon) \big) \Big]. \quad (2)$$

$\rho = \pi_\psi(a \mid o_t) / \pi_{\text{old}}(a \mid o_t)$ is the probability ratio between the current policy and the policy π_{old} that collected the rollout. $\bar{A}$ is $\hat{A}$ standardized within the mini-batch, and ϵ is the clipping range. $\mathcal{L}_V = \frac{1}{2}\mathbb{E}\big[(V_\omega(\text{aug}(o_t)) - \hat{R})^2 \big]$ is the value loss, weighted by c_v. $\beta(t)$ is the BC weight. It decays linearly with the environment step t to a small floor. $\kappa \in \{0, 1\}$ is a KL gate. It equals 1 until the approximate KL divergence between π_ψ and π_{old} on some mini-batch exceeds a threshold, and 0 for the rest of that update. After that point the actor and the BC anchor are frozen, while the value function keeps training. No entropy bonus is used. ρ is evaluated on unaugmented frames, and $\log \pi_{\text{old}}$ is recomputed at the start of each update, so that both policies are compared on the same input. Augmentation is applied only to the value and BC branches. The proprioceptive statistics are held fixed during each rollout and update and refreshed afterwards. The actor and critic parameters use separate learning rates.

Reinforcement Learning with Prior Data (RLPD). RLPD is built on soft actor-critic (SAC), an off-policy method that trains a stochastic policy to maximize the return plus an entropy bonus. Relative to SAC, RLPD makes three changes. First, it uses symmetric sampling: every mini-batch is drawn half from the demonstration dataset $\mathcal{D}_{\text{demo}}$ and half from the online replay buffer $\mathcal{D}_{\text{replay}}$, and it applies LayerNorm to efficiently prevent overestimation. Second, it replaces SAC's two critics with an ensemble of LayerNorm critics, and its target takes the minimum over a random subset of heads. Third, it allows several gradient updates per environment step. We use RLPD without algorithmic changes and adapt only the input and action pipeline.

Observations and image encoders are the same as PPO. However, a single encoder f_ϕ with parameters ϕ is used, giving the state feature $s_t = [f_\phi(o_t), \, n(p_t)]$. The policy does not output absolute joint targets directly. Instead it outputs a bounded increment $u_t \in [-1, 1]^{|A|}$, and the command sent to the robot is

$$a_t = \text{clip}\big(a_{t-1} + \delta \, u_t, \; a_{\text{low}}, \; a_{\text{high}} \big). \quad (3)$$

Here δ is the maximum per-step change in radians, and $[a_{\text{low}}, a_{\text{high}}]$ are the joint limits. The previous command a_{t-1} is appended to p_t so that the state remains Markovian. At $t = 0$, a_{-1} is set to the mean first-frame demonstration action. Both the actor and the critic operate on the increment u_t. Demonstration actions are converted to the same increments before training. The actor $\pi_\psi(u \mid s)$ is a tanh-squashed Gaussian whose mean and standard deviation both depend on s. The critic is an ensemble of N Q-functions $Q_{\theta_k}(s, u)$, $k = 1, \ldots, N$, each an MLP with LayerNorm. Each Q_{θ_k} has a target copy $Q_{\bar{\theta}_k}$. The reward is the success flag shifted by -1, $\tilde{r}_t = \text{succ}_t - 1 \in \{-1, 0\}$, a per-step cost that stops on success. The termination flag d_t equals succ_t. Time-limit truncations are therefore not terminal and are bootstrapped. Under this reward, Q equals the negative discounted number of steps to success.

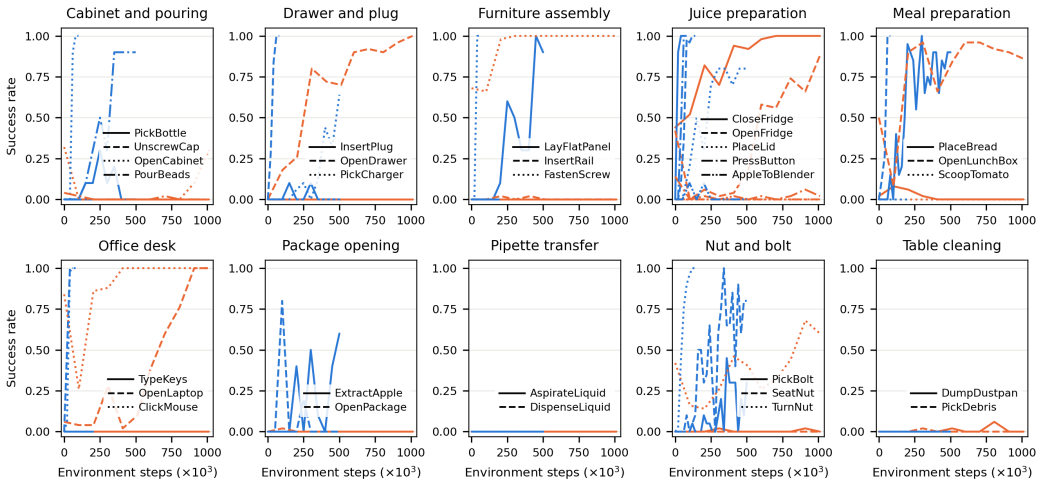


Figure 5: Training curves of the RL baselines on the 30 stage-level tasks, grouped by long-horizon task. Orange curves are PPO and blue curves are RLPD; line styles distinguish the stage-level tasks within each long-horizon task. The point at 0 steps for PPO is the policy after BC warm-up. PPO is trained for a fixed budget of 10^6 environment steps on every stage-level task. RLPD is trained for up to 5×10^5 environment steps and stopped early once the evaluation success rate reaches 1.0 in two consecutive evaluations or stays at 0 for ten consecutive evaluations, so each RLPD curve ends where its run was stopped.

Each mini-batch $\mathcal{B}$ contains B transitions $(o_t, u_t, \tilde{r}_t, d_t, o_{t+1})$, half from each source, and both halves are treated identically. The critic is trained as in SAC:

$$\mathcal{L}_Q = \frac{1}{NB} \sum_{k=1}^{N} \sum_{t \in \mathcal{B}} \big(Q_{\theta_k}(s_t, u_t) - y_t\big)^2, \qquad y_t = \tilde{r}_t + \gamma\,(1 - d_t) \min_{k \in \mathcal{K}} Q_{\bar{\theta}_k}(s_{t+1}, u'), \tag{4}$$

where $u' \sim \pi_\psi(\cdot \mid s_{t+1})$ and $\mathcal{K}$ is a random subset of $M < N$ heads. Unlike SAC, the target contains no entropy term. The next-state feature s_{t+1} is computed by the current encoder under stop-gradient $\mathrm{sg}(\cdot)$, which blocks gradients from flowing back through it. Hence y_t serves as a fixed regression target, and no separate target encoder is kept. $\mathcal{L}_Q$ is the only loss that trains the encoder. The actor receives the detached feature $\bar{s}_t = [\mathrm{sg}(f_\phi(o_t)),\ n(p_t)]$ and minimizes

$$\mathcal{L}_\pi = \frac{1}{B} \sum_{t \in \mathcal{B}} \Big[\alpha \log \pi_\psi(\tilde{u}_t \mid \bar{s}_t) - \frac{1}{N} \sum_{k=1}^{N} Q_{\theta_k}(\bar{s}_t, \tilde{u}_t)\Big], \tag{5}$$

where $\tilde{u}_t$ is a reparameterized sample from $\pi_\psi(\cdot \mid \bar{s}_t)$. The temperature α is tuned automatically. The target critics track the online ones by Polyak averaging.

Action Chunking with Transformers (ACT). The visual observation of ACT consists of 256×256 RGB images from a fixed front camera and a task-relevant wrist camera. For each camera, we use the frames at temporal offsets $\{0, 2, 4\}$, resulting in six visual inputs per policy query. The proprioceptive input contains only the current robot joint positions; no privileged object state is provided. Both methods output absolute target joint positions under joint-position control. The state and action dimensions are 29 for single-arm tasks, comprising 7 arm joints and 22 hand joints, and 58 for bimanual tasks. A shared ImageNet-pretrained ResNet-18 converts each camera image into spatial feature tokens. The image tokens and an encoded proprioceptive-state token are fused by a Transformer to obtain the observation context. A conditional-VAE posterior encoder takes the context and demonstrated action sequence as input, while the decoder predicts a 25-step action chunk.

Let $\hat{\mathbf{a}}^{\mathrm{post}}$ and $\hat{\mathbf{a}}^0$ denote the action chunks decoded from the sampled posterior latent and the zero latent, respectively. The training objective is

$$\mathcal{L}_{\mathrm{ACT}} = \mathcal{L}_{\mathrm{rec}}\left(\hat{\mathbf{a}}^{\mathrm{post}}, \mathbf{a}\right) + \lambda_0 \mathcal{L}_{\mathrm{rec}}\left(\hat{\mathbf{a}}^0, \mathbf{a}\right) + \lambda_{\mathrm{KL}} D_{\mathrm{KL}}\left(q(\mathbf{z} \mid \mathbf{o}, \mathbf{a}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\right), \tag{6}$$

where $\mathcal{L}_{\text{rec}}$ is a masked ℓ_1 loss that excludes padded actions. We compute the reconstruction loss separately for the arm and hand action groups, using weights of 2 and 1, respectively. For bimanual tasks, the same weighting is applied independently to the two arms and hands. We use $\lambda_0 = 1$ and $\lambda_{\text{KL}} = 10^{-3}$. The zero-latent term directly trains the inference path instead of relying only on the sampled posterior latent.

At inference time, the latent variable is fixed to zero and the policy predicts a new action chunk at every control step. All predictions corresponding to the current step are combined using temporal ensembling. A prediction generated Δt steps earlier is assigned weight

$$w(\Delta t) \propto \exp(-\alpha \Delta t), \tag{7}$$

where $\alpha = 0.05$. This allows the controller to combine the temporal consistency of previous predictions with the responsiveness of newly generated commands.

Diffusion Policy (DP). Our DP implementation uses the same visual history, proprioceptive state, and absolute joint-position action representation as ACT. A shared ImageNet-pretrained ResNet-18 with Group Normalization encodes each image into a 128-dimensional feature. The features from all camera and temporal inputs are concatenated with the current joint positions and used as the global condition of a temporal U-Net. The model generates a 16-step action sequence.

During training, the actions are normalized to $[-1, 1]$ using the per-dimension minimum and maximum values computed from the training split. Given a clean action sequence $\mathbf{a}^0$, a diffusion timestep k, and Gaussian noise $\boldsymbol{\epsilon}$, the noisy action sequence is

$$\mathbf{a}^k = \sqrt{\bar{\alpha}_k}\, \mathbf{a}^0 + \sqrt{1 - \bar{\alpha}_k}\, \boldsymbol{\epsilon}. \tag{8}$$

The temporal U-Net is trained to predict the added noise:

$$\mathcal{L}_{\text{DP}} = \mathbb{E}_{k,\boldsymbol{\epsilon}} \left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta \left(\mathbf{a}^k, k, \mathbf{o} \right) \right\|_2^2 \right]. \tag{9}$$

Losses on padded action steps are masked out. We maintain an exponential moving average of both the visual encoder and the temporal U-Net and use the averaged model for evaluation.

At inference time, a 16-step action sequence is initialized from Gaussian noise and generated using 100 reverse DDPM steps with a cosine noise schedule. The visual observations are encoded once for each generated chunk. The controller executes the first four actions and then generates a new sequence conditioned on the latest observations. Therefore, DP operates as a receding-horizon controller with an execution horizon of four steps.

GR00T N1.7. Observation and action adaptation. We fine-tune through GR00T's own training entry point and evaluate the resulting policy in process, inside the same closed-loop simulator used by every other baseline. Single-arm tasks use one external and one wrist RGB view; the bimanual task uses one external view and one wrist view per arm. Images are stored at 256×256 and aligned in time with the corresponding robot states and action targets. The released demonstrations are recorded without images, so each frame is re-rendered by restoring the recorded simulator state; we verified this path against per-frame environment resets and found identical pixels and states.

Raw observations may contain both robot and task-environment information. We therefore read the robot's actuated joint positions explicitly and reorder them from the articulation's joint order into the controller's order, rather than truncating the raw state vector; a joint that the controller does not drive raises an error instead of being silently included. Single-arm Sharpa control comprises seven arm joints and 22 hand joints. Bimanual control comprises 58 targets ordered as the joint-position controller concatenates its sub-controllers (left arm, right arm, left hand, right hand), taken from the robot and controller definitions rather than assuming a concatenation of two single-arm state vectors. The action head admits up to 132 dimensions, so the 29- and 58-dimensional layouts are used without architectural widening; each task is registered as a new embodiment whose state and action encoders are randomly initialized.

State and action vectors are declared as named joint groups rather than as one flat vector, and each group carries its own action representation: arm groups are predicted as offsets from the current configuration, hand groups as absolute targets. Normalization statistics are computed from the training

Table 10: Training configurations of PPO and RLPD. Both methods use the same observation interface (two 128×128 RGB cameras plus proprioception) and the same demonstrations (100 VR teleoperation episodes per stage-level task). "–" means not applicable.

Configuration	**PPO**	**RLPD**
Observations and actions		
Image resolution	128×128, base and wrist camera	128×128, base and wrist camera
Proprioception	q, $\dot{q}$, EE pose	q, $\dot{q}$, EE pose, a_{t-1}
Action	Absolute joint targets, 29-D	Joint-target increments, 29-D
Action bound	$\tanh$, demo range $+10\%$	$0.2\, u_t$, $u_t \in [-1, 1]$
Policy distribution	Gaussian, $\log \sigma_0 = -3.5$	$\tanh$-Gaussian, $\log \sigma \in [-5, 2]$
Networks		
Visual encoder	4-layer CNN, from scratch (32-64-128-256)	4-layer CNN, from scratch (32-64-128-256)
Latent per camera	128	64
Encoder gradient	Actor and critic	Critic only
MLP hidden layers	4×512	3×256
Critic ensemble	–	5 heads, min over 2
Image augmentation	Random shift, pad 4	Random shift, pad 4
Reward and objective		
Reward	$\mathbf{1}[\text{success}]$	$\mathbf{1}[\text{success}] - 1$
Discount γ	0.99	0.999
GAE λ	0.95	–
Clip coefficient / target KL	0.2 / 0.2	–
Value / entropy coefficient	0.5 / 0	–
Entropy temperature	–	Auto-tuned
Target network τ	–	0.005
Optimization		
Optimizer	Adam	Adam
Learning rate	3×10^{-5} actor, 3×10^{-4} critic	3×10^{-4}
Gradient clipping	0.5	10
Parallel environments	64	1
Batch size	64×200, 32 minibatches	256 (50% demo)
Epochs / UTD	1 epoch	UTD 1
Replay buffer	–	2×10^5
Random warm-up	–	5,000 steps
Demonstration usage		
Demonstrations per stage-level task	100	100
BC warm-up	20,000 steps, batch 256	–
BC anchor coefficient	$1.0 \to 0.05$ over 2.5×10^6	–
Demo-state reset probability	$1.0 \to 0.1$ over 3.5×10^6	–
Reverse-curriculum window	$20\% \to 100\%$ over 2.5×10^6	–
Curriculum pacing	$\times 3$ / $\times 1$ / $\times 0.5$ at success ≥ 0.8 / ≥ 0.5 / < 0.5	–
Demo sampling	–	50% of every batch
Budget and evaluation		
Environment steps	1×10^6	5×10^5
Episode length	500	500
In-training evaluation	50 episodes every 10^5 steps	20 episodes every 20–50k steps
Reported evaluation	Best checkpoint, 50 episodes	Best checkpoint, 50 episodes
Seed	1	1

dataset and stored in the dataset metadata alongside the group declaration. At inference, outputs are inverse-transformed into absolute joint-position targets and clipped to the controller's bounds.

Demonstrations and language conditioning. Each stage-level task has 100 demonstrations, converted to the LeRobot v2.1 layout with the joint-group declaration written to the dataset metadata. Every demonstration of a stage-level task carries the same short imperative instruction (e.g. "open the cabinet door"), stored in the dataset and read by the training loader; the identical sentence con-

Table 11: Implementation and training configurations of ACT and Diffusion Policy. Parameters shared by both methods are listed in the same row.

Configuration	ACT	Diffusion Policy
Visual observations	Front and wrist RGB images at offsets $\{0, 2, 4\}$	Front and wrist RGB images at offsets $\{0, 2, 4\}$
Image resolution	256×256	256×256
Proprioceptive state	Current robot joint positions	Current robot joint positions
Action representation	Absolute target joint positions	Absolute target joint positions
State/action dimension	29 (single-arm), 58 (bimanual)	29 (single-arm), 58 (bimanual)
Visual encoder	ImageNet-pretrained ResNet-18	ImageNet-pretrained ResNet-18 with GroupNorm
Action normalization	Per-dimension mean and standard deviation	Per-dimension minimum and maximum
Prediction horizon	25 steps	16 steps
Hidden dimensions	Transformer hidden size 256; latent size 32	U-Net channels $(256, 512, 1024)$
Transformer configuration	4 heads; 2 posterior, 2 fusion, and 4 decoder layers	–
Diffusion configuration	–	100 DDPM steps; cosine noise schedule
Reconstruction objective	Masked group-weighted ℓ_1	Masked noise-prediction MSE
Arm/hand loss weights	$2:1$ for each arm–hand pair	Uniform over action dimensions
Zero-latent loss weight λ_0	1.0	–
KL weight λ_{KL}	10^{-3}	–
Batch size	16	16
Training updates	20,000	20,000
Optimizer	AdamW	AdamW
Training random seed	1	1
Learning rate	10^{-4}	10^{-4}
Weight decay	10^{-4}	10^{-4}
Learning-rate schedule	Constant	500-step warm-up followed by cosine decay
Gradient clipping	Maximum norm 10	Maximum norm 10
Train/validation split	90%/10% by episode	90%/10% by episode
Inference model	Zero-latent prediction	Exponential-moving-average model
Inference strategy	Temporal ensemble at every control step	Execute 4 actions and replan
Temporal-ensemble decay	0.05	–
Checkpoint interval	1,000 updates	1,000 updates
Stage-level evaluation seeds	50000–50049	50000–50049

ditions the policy at evaluation time. We record the code commit and the instruction table version and keep them fixed within each experiment.

Training. Each of the 30 stage-level tasks is fine-tuned independently from the pretrained GR00T N1.7 checkpoint using its own demonstrations. We retain the flow-matching objective and update all parameters, including the visual encoder and the language backbone, without freezing any module and without LoRA. Training images are augmented with the entry point's default random crop (0.95 of each side, resized back to 256×256) and color jitter; evaluation uses the center crop. Each stage-level task uses 12,000 optimizer updates at a global batch size of 32, realized on a single GPU as a micro-batch of two with 16 gradient-accumulation steps, with a single training seed (42) per task. Runs exceeding the cluster's job time limit are split into segments that resume from the latest checkpoint, restoring optimizer state, learning-rate schedule and step count. Checkpoints are written every 1,000 updates; we always evaluate the final checkpoint and never select a checkpoint on evaluation performance. The configuration is summarized in Table 12.

Long-horizon models. For the long-horizon evaluation we additionally fine-tune one policy per long-horizon task on the pooled demonstrations of all its stage-level tasks (100 per stage, i.e. 200–500 episodes), each demonstration keeping its own stage instruction. Architecture, hyper-parameters and budget are identical to the stage-level models (12,000 updates), so a long-horizon model receives

the same compute as a single stage-level model. For *Package opening*, whose two stages use different embodiments (bimanual OpenPackage, right-arm ExtractApple), the pooled dataset uses the 58-dimensional bimanual layout with the external and right-wrist views; the single-arm episodes are padded with the bimanual robot's initial left-arm configuration, and only the right-arm outputs are executed on that stage. At evaluation the model runs one uninterrupted episode in the long-horizon environment and the instruction is switched to the next stage as soon as the environment reports the current stage complete.

Inference and evaluation. The policy predicts a 40-step action chunk with four denoising steps and replans from the latest observation after executing the first eight actions. Each stage-level task is evaluated for 50 episodes using the final checkpoint under the task's registered episode limit. Episode i ($i = 0, \ldots, 49$) resets the environment with seed $10001 + i$, so all 30 stage-level tasks are evaluated on the same 50 initial states of their respective layouts; the evaluation process is seeded with 10001 before the first episode (`random`, NumPy and PyTorch), which also fixes the initial noise of the flow-matching sampler. The same 50 seeds are used for the three generalization scenes and for the long-horizon tasks, where the long-horizon environment is reset with seed $10001+i$ once per episode. The instruction is the fixed sentence used in training. Episode seeds, episode lengths and task outcomes are recorded. An episode counts as a success if the environment's native success condition holds at any step.

A.7 Dataset Statistics and Analysis

A.7.1 Hand Usage

Table 13 gives the three statistics summarised in Figure 4 for every stage-level task. Each stage-level task pools the frames of its 100 released episodes up to the first success. For the bimanual stage-level task (OpenPackage) the two hands are analyzed separately and averaged. The category rows are the values plotted in the main figure and quoted in Section 4.1. Category values weight stage-level tasks equally.

In Grasping and Insertion, the contact sits on the fingertip pads, led by the thumb pad (86% and 82% of contact frames) with the middle and index pads next; the ring pad joins in a third of the frames and the little finger and palm almost never. Twisting spreads contact over fewer frames and more links per finger, as the fingers release and re-contact the part. Pressing is the one category without thumb contact: the index pad and distal phalanx carry the button and key presses, while the middle and ring pads (37% and 32%) and the palm (25%) come from holding the pipette. Scooping and pouring wraps the whole thumb around the tool, with all three thumb links in contact for a third of the frames or more. Opening and closing shows the lightest and most evenly spread contact. Joint excursion is carried mainly by the knuckle flexion joints of the index, middle and ring finger, which move through 40–52% of their range in grasping, twisting and scooping. Table 13 gives the timesteps to success, contact ratio and fingers in contact for every stage-level task; the category rows are the values plotted in Figure 4.

A.7.2 Episode Diversity

Table 14 measures how much the 100 episodes of a stage-level task differ in the motion of the hand. The palm position is obtained by forward kinematics from the recorded joint positions. Across stage-level tasks, the mean palm path length ranges from 0.35 m to 2.04 m, and the palm paths of two episodes are 3.3–10.2 cm apart on average. The coefficient of variation of the time to success is 0.10–0.36.

A.7.3 Open-loop Replay

Every accepted demonstration is replayed open loop. The recorded initial state is restored in a fresh environment, the recorded action sequence is executed step by step, and the episode counts as a replay success if the success condition becomes true at any step. The replay uses the same simulator build as the recording. Over the 30 stage-level tasks, 3222 of the 4112 accepted episodes (78%) replay to success. The released 100 episodes of each stage-level task are drawn from the replay-successful ones (Appendix A.5).

Table 12: **GR00T N1.7 configuration.** Values not listed as arguments of our training script are the defaults of the released GR00T fine-tuning entry point.

Configuration	Setting
Pretrained model	GR00T N1.7-3B
Training organization	One independent policy per stage-level task (30 stage-level tasks)
Demonstrations	100 per stage-level task
Language instruction	One fixed instruction per stage-level task, shared by training and evaluation
Fine-tuning	All parameters
Training objective	Flow matching
Vision encoder	Trainable
LoRA	Disabled
Embodiment	New embodiment; state and action encoders randomly initialized
Single-arm visual inputs	External RGB and wrist RGB
Bimanual visual inputs	External RGB and left/right wrist RGB
Stored image resolution	256×256 per view
Proprioception	Joint positions in controller order
Physical state/action dimensions	29 (single-arm); 58 (bimanual)
Model state/action capacity	132
Action representation	Arm relative, hand absolute; executed as absolute targets
State/action grouping	Named joint groups (arm, hand; both sides if bimanual)
State/action normalization	Min–max on training-set 1st/99th percentiles, clipped to $[-1, 1]$
State dropout	0.2
Image augmentation	Random crop of 0.95 of each side (230×230) resized back to 256×256; color jitter (brightness 0.3, contrast 0.4, saturation 0.5, hue 0.08); no rotation. Evaluation uses the center crop
Optimizer	AdamW
AdamW β_1, β_2	0.9, 0.999
Optimizer updates	12,000 per stage-level task
Global batch size	32
Micro-batch $\times$ accumulation	2×16 on one GPU
Peak learning rate	4×10^{-5}
Learning-rate schedule	Linear warmup (5% of updates, 600 steps) then cosine decay
Weight decay	1×10^{-5}
Gradient-norm clipping	1.0
Precision	BF16
Gradient checkpointing	Disabled
Hardware	$1\times$ A100-80GB
Training seed	42 (one run per stage-level task; no seed sweep)
Action prediction horizon	40 steps
Actions executed per query	8
Inference denoising steps	4
Physics backend	PhysX CPU
Episode limit	Task's registered horizon
Evaluation checkpoint	Update 12,000
Evaluation episodes	50 per stage-level task
Evaluation seeds	Environment seeds 10001–10050, one per episode; the same 50 seeds for every stage-level task, every generalization scene and the long-horizon protocol; the evaluation process is seeded with 10001

Table 13: Timesteps to success, contact ratio and fingers in contact for every stage-level task, grouped by motion category. Timesteps: control steps (20 Hz) to first success, median over the 100 episodes with the interquartile range in parentheses; category rows give the range of the stage-level task medians. Contact ratio: fraction of frames, up to first success, in which a finger or the palm touches the primary object of the stage-level task. Fingers: number of distinct fingers in contact, averaged over hand-contact frames. Stage-level task values pool their 100 episodes. Category rows weight stage-level tasks equally.

Stages	Timesteps	Contact ratio	Fingers
Opening and closing	**60–162**	**0.52**	**2.3**
OpenLunchBox	99 (93–109)	0.78	1.6
OpenCabinet	110 (99–120)	0.58	3.2
OpenDrawer	103 (91–120)	0.59	3.5
OpenFridge	162 (134–183)	0.63	3.6
CloseFridge	60 (56–67)	0.45	1.9
OpenLaptop	101 (92–107)	0.37	1.0
OpenPackage	156 (134–208)	0.23	1.1
Grasping and placement	**118–376**	**0.54**	**3.1**
PickBottle	208 (186–225)	0.48	3.0
PickCharger	201 (173–238)	0.54	3.4
PickBolt	118 (105–137)	0.60	2.1
PlaceBread	147 (131–173)	0.59	3.7
ExtractApple	248 (223–280)	0.48	3.7
AppleToBlender	249 (221–270)	0.52	3.8
LayFlatPanel	148 (130–184)	0.69	2.4
PickDebris	376 (350–412)	0.40	2.5
Insertion and assembly	**155–248**	**0.68**	**2.7**
InsertPlug	224 (190–275)	0.80	3.5
InsertRail	248 (192–306)	0.55	1.9
SeatNut	155 (140–180)	0.61	2.0
PlaceLid	162 (140–182)	0.77	3.5
Twisting and screwing	**41–310**	**0.47**	**2.1**
UnscrewCap	310 (250–356)	0.60	2.5
TurnNut	158 (133–209)	0.48	2.1
FastenScrew	41 (38–47)	0.34	1.6
Pressing	**62–474**	**0.40**	**1.9**
PressButton	74 (64–86)	0.13	1.0
ClickMouse	62 (58–68)	0.07	1.0
TypeKeys	440 (398–480)	0.22	1.0
AspirateLiquid	326 (302–365)	0.77	3.2
DispenseLiquid	474 (432–521)	0.80	3.4
Scooping and pouring	**145–332**	**0.71**	**3.1**
ScoopTomato	332 (288–369)	0.81	3.0
PourBeads	193 (168–213)	0.65	4.4
DumpDustpan	145 (138–154)	0.67	2.0

Table 15 lists the replay success per stage-level task next to the three contact statistics used below. Replay success is high for Opening and Closing (98%) and lowest for Insertion and Assembly (65%) and Scooping and Pouring (74%). Within the latter two categories it varies widely: PlaceLid and InsertPlug replay in 58% and 62% of episodes, ScoopTomato in 38%, while DumpDustpan reaches 94%.

Across the 30 stage-level tasks, replay success is negatively correlated with the contact ratio (Spearman $\rho = -0.58$, $p < 0.001$), with the number of contact events per episode ($\rho = -0.51$, $p = 0.004$) and with the number of fingers in contact ($\rho = -0.44$, $p = 0.014$).

Stage-level tasks involving more sustained or repeated contact therefore tend to have lower open-loop replay success. We observe trajectory deviations from the recorded rollouts shortly after replay begins, even though the recorded initial state and action sequence are restored. Together, these

Table 14: Diversity of the released demonstrations, per stage-level task. Path length: length of the palm path from the first frame to first success, mean over the 100 episodes. Pairwise distance: mean top-view distance between the palm paths of two episodes after resampling both to 50 points in normalised time, averaged over all episode pairs. Time CV: coefficient of variation of the timesteps to first success. For the bimanual stage-level task (OpenPackage) the hand values are given as left / right.

Stages	Path length (m)	Pairwise distance (cm)	Time CV
Opening and closing			
OpenLunchBox	0.53	3.9	0.13
OpenCabinet	0.86	4.9	0.15
OpenDrawer	0.60	3.9	0.20
OpenFridge	1.11	6.9	0.21
CloseFridge	1.38	8.7	0.13
OpenLaptop	0.80	5.1	0.10
OpenPackage	0.72 / 0.65	6.6 / 6.4	0.36
Grasping and placement			
PickBottle	1.40	7.9	0.14
PickCharger	1.90	8.9	0.20
PickBolt	0.76	7.2	0.18
PlaceBread	0.99	5.5	0.24
ExtractApple	1.75	7.0	0.17
AppleToBlender	1.93	8.1	0.12
LayFlatPanel	1.29	7.5	0.26
PickDebris	1.57	6.2	0.12
Insertion and assembly			
InsertPlug	1.08	7.5	0.28
InsertRail	1.13	10.2	0.29
SeatNut	0.88	6.8	0.20
PlaceLid	0.89	5.7	0.18
Twisting and screwing			
UnscrewCap	1.40	9.6	0.23
TurnNut	0.88	7.1	0.34
FastenScrew	0.50	5.1	0.29
Pressing			
PressButton	0.41	4.3	0.26
ClickMouse	0.35	3.3	0.12
TypeKeys	1.37	4.7	0.13
AspirateLiquid	1.62	4.9	0.15
DispenseLiquid	2.04	7.0	0.13
Scooping and pouring			
ScoopTomato	1.98	7.8	0.15
PourBeads	1.13	8.5	0.17
DumpDustpan	0.99	4.6	0.11

results are consistent with contact-rich motions being more sensitive to small state or simulator differences under open-loop replay.

A.8 Generalization Evaluation Scenes

The default training scene and the varied scenes for generalization evaluation are visualized in Figure 6.

A.9 Long-horizon Performance

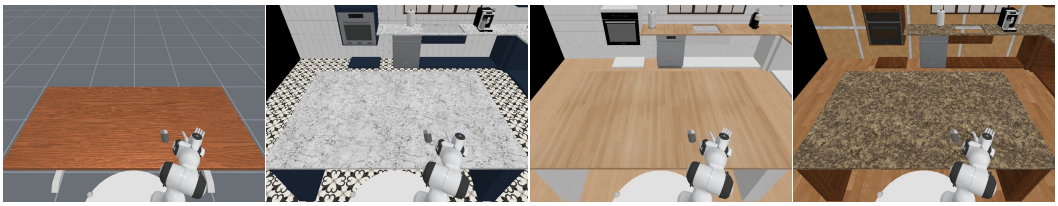
We evaluate all five baselines on the 10 long-horizon tasks. For the single-task baselines, we reuse the learned stage-level policies without additional training and switch policies when the success condition of the current stage is met. For the VLA baseline, we train one model for each long-horizon

Table 15: Open-loop replay success of the accepted demonstrations, per stage-level task and motion category, next to the three contact statistics it correlates with. Replay: the recorded actions are replayed from the recorded initial state in a fresh simulator instance; an episode counts as replayable if the success condition becomes true at any step (replayable/accepted episodes in parentheses). Contact ratio and fingers: as in Table 13. Events: number of separate hand-contact intervals with the primary object per episode, up to first success, mean over episodes. Category rows weight stage-level tasks equally.

Stages	Replay success	Contact ratio	Events	Fingers
Opening and closing	**98%**	**0.52**	**2.1**	**2.3**
OpenLunchBox	99% (110/111)	0.78	2.2	1.6
OpenCabinet	99% (100/101)	0.58	1.9	3.2
OpenDrawer	100% (110/110)	0.59	1.9	3.5
OpenFridge	93% (107/115)	0.63	2.0	3.6
CloseFridge	100% (104/104)	0.45	2.2	1.9
OpenLaptop	100% (109/109)	0.37	2.7	1.0
OpenPackage	97% (102/105)	0.23	2.0	1.1
Grasping and placement	**84%**	**0.54**	**3.2**	**3.1**
PickBottle	83% (110/132)	0.48	3.3	3.0
PickCharger	94% (107/114)	0.54	2.5	3.4
PickBolt	74% (107/145)	0.60	2.6	2.1
PlaceBread	84% (102/122)	0.59	2.4	3.7
ExtractApple	92% (104/113)	0.48	3.6	3.7
AppleToBlender	83% (106/127)	0.52	2.3	3.8
LayFlatPanel	85% (102/120)	0.69	2.1	2.4
PickDebris	74% (101/136)	0.40	6.4	2.5
Insertion and assembly	**65%**	**0.68**	**3.5**	**2.7**
InsertPlug	62% (101/163)	0.80	2.2	3.5
InsertRail	62% (110/177)	0.55	6.4	1.9
SeatNut	80% (105/132)	0.61	2.8	2.0
PlaceLid	58% (116/199)	0.77	2.7	3.5
Twisting and screwing	**87%**	**0.47**	**4.7**	**2.1**
UnscrewCap	73% (111/153)	0.60	7.5	2.5
TurnNut	96% (118/123)	0.48	4.9	2.1
FastenScrew	94% (119/127)	0.34	1.8	1.6
Pressing	**83%**	**0.40**	**3.1**	**1.9**
PressButton	100% (100/100)	0.13	1.3	1.0
ClickMouse	99% (114/115)	0.07	1.1	1.0
TypeKeys	97% (111/115)	0.22	5.9	1.0
AspirateLiquid	81% (101/125)	0.77	3.6	3.2
DispenseLiquid	39% (108/280)	0.80	3.8	3.4
Scooping and pouring	**74%**	**0.71**	**2.0**	**3.1**
ScoopTomato	38% (118/311)	0.81	2.1	3.0
PourBeads	89% (107/120)	0.65	2.0	4.4
DumpDustpan	94% (102/108)	0.67	2.0	2.0

task using demonstrations from all of its constituent stage-level tasks and switch the instruction between stages. The environment is initialized only at the beginning of the full task and is not reset between stages. An episode succeeds only if all required stages are completed in order and the final success condition of the full task is satisfied within the evaluation horizon.

Table 16 reports the resulting end-to-end success rates.



(a) Training (b) Generalization 1 (c) Generalization 2 (d) Generalization 3

Figure 6: **Evaluation scenes for generalization.** Training and three held-out scene settings used to evaluate visual and environmental generalization across the 30 stage-level tasks. The scenes span three kitchen environments while preserving the task-specific objects and success conditions.

Table 16: **Long-horizon performance.** End-to-end success rates (%, ↑) on 10 long-horizon tasks, with 50 evaluation episodes per task. An episode succeeds only if all required stages are completed in order within the evaluation horizon.

Task	PPO	RLPD	ACT	DP	GR00T
Meal preparation	0	0	0	0	0
Cabinet and pouring	0	0	0	0	0
Drawer and plug	0	0	0	0	0
Nut and bolt	0	0	0	0	0
Office desk	0	0	0	0	0
Furniture assembly	0	0	8	0	0
Pipette transfer	0	0	0	0	0
Juice preparation	0	0	0	0	0
Package opening	0	0	4	0	0
Table cleaning	0	0	0	0	0
Average	0	0	1.2	0	0